

## Java

### a

**Array:** tipo de dato estructurado, permite almacenar un conjunto de datos homogéneo, es decir, todos del mismo tipo y relacionados. Cada uno de los elementos pueden ser de tipo simple como caracteres, entero, real, o de tipo compuesto, estructurado como son otros arrays u objetos.

Una vez declarado el tamaño de un array, éste no puede ser modificado. El índice inicial de los arrays en Java es 0.

#### Declaración

```
<tipo dato> [ ] <nombre_array>;  
<tipo dato> <nombre_array> [ ];  
<tipo dato> [ ] <nombre_array> [ ];
```

Ej.

```
int [ ] arrayEnteros; //Declaración array de enteros  
string myStringArray [ ]; //Declaración array de strings  
char [ ] myCharArray [ ]; //Declaración array de chars
```

#### Inicializar

Por tamaño:

```
<tipo dato> [ ] <nombre_array> = new <tipo dato>[ tamaño];
```

Ej. `int[ ] arrayEnteros = new int[ 5 ];`

Con valores:

```
<tipo dato> [ ] <nombre_array> = { val1, val2, ... };
```

Ej. `String[ ] arrayNombres = { "Juan", "María", "Susana" };`

Recuperar o asignar un valor a una posición uso del índice.

Recuperar:

```
<tipo dato> <nombre_variable> = <nombre_array> [ índice ];
```

Ej. `int numero = arrayEnteros [ 2 ];`

### b

**Bloques:** Conjunto de sentencias, que están delimitados por llaves ({}).

### c

## Java

**Casting: (función Casting):** transforma un **dato primitivo** de un tipo a otro en función del tamaño que ocupa. La conversión puede ser de dos formas:

**Casting implícito:** cuando el **contenedor es más grande** que el valor a almacenar.

byte-short-char-int-long-float-double

**Casting explícito:** cuando el **contenedor es más pequeño** que el valor a almacenar. En este caso, puede que haya pérdida de datos.

double-float-long-int-char-short-byte

**Clase:** Todo programa Java se llama clase. Plantilla que define la forma que tendrán los objetos creados en la misma. Es decir el tipo de características (atributos) y los comportamientos (métodos) comunes.

**Atributos:** Almacenan **características** de un objeto (lo que se sabe de un objeto; ej. color, longitud). El nombre por convención se escribe empezando por **minúscula**, y delante de cada atributo es necesario indicar el **tipo de dato** seguido del **nombre del atributo**.

### Datos

**Primitivo:** int (números enteros), float (decimales de hasta 7 dígitos), double (decimales más precisos de hasta 16 dígitos).

**Complejo:** String (texto o cadenas de texto), Date(fecha). También podemos crear nuestros propios tipos complejos.

**Métodos:** Representan el **comportamiento** de un objeto (lo que hace un objeto; ej. acelerar, frenar).

### Declaración

Mediante la palabra reservada **class** (puede ir precedida de un modificador de acceso) seguida de su **nombre**.

### Modificadores

**public:** atributo público, se puede acceder a la clase desde **cualquier lugar del programa**.

**protected:** son accesibles directamente sólo desde la propia clase, de las subclases de esta (herencia) y las clases que se encuentran dentro del mismo paquete es decir se puede acceder a la clase desde el **paquete o subclase** de esta.

**private:** atributo privado no se podrá acceder directamente a él desde otra clase distinta. Por tanto, será necesario recurrir a algún método que nos proporcione o nos modifique su valor. sólo se puede acceder a la clase desde esta **misma**.

```
[modificadores] class NombreClase{
```

### Implementación

Incluye los atributos y métodos de la clase, después de su declaración.

## Java

```
public class Coche{

    //declaración de atributos
    private String color;
    private double longitud;

    //declaración de métodos
    public void acelerar(){};
    public void frenar(){};
}
```

### Atributos

#### Comentario:

##### Comentarios de una sola línea

```
// Esto es un comentario
```

##### Comentarios multilínea

```
/* Esto es un comentario
de una o más líneas */
```

##### Comentarios de documentación para la herramienta Javadoc

```
/** Esto es un comentario para javadoc */
```

**Clase Wrapper:** envoltorio, permite usar tipos primitivos (int, char, boolean, etc.) como objetos.

**int** → **Integer**

**double** → **Double**

**char** → **Character**

**boolean** → **Boolean**

Permiten usar métodos útiles y almacenar valores primitivos en colecciones que sólo aceptan objetos (como `ArrayList<Integer>`).

```
int num = 5;
Integer objNum = Integer.valueOf(num); // Boxing (convertir primitivo a objeto)
int num2 = objNum.intValue();          // Unboxing (objeto a primitivo)
```

**Constante:** Objeto que no experimenta cambios durante todo el desarrollo del algoritmo.

#### Declaración

Definir a nivel de clase (global)

Hacer uso de 2 palabras reservadas: **static** y **final**

Nombres en mayúsculas.

```
static final <tipo_dato> <NOMBRE CONSTANTE> = valor;
```

## Java

### d

#### Datos:

**Primitivos o simples:** entidades elementales como números, caracteres o Verdadero/Falso.

- **Int:** números de tipo entero, como años o días de la semana.

```
int nAño = 2005;
```

- **Float:** números de tipo decimal de hasta 7 dígitos; como altura o peso.

```
float nAltura = 74.54;
```

- **Double:** números de tipo decimal muy precisos, hasta 16 dígitos; como área o longitud.

```
double nLongitud = 2500000.5497;
```

- **Char:** variables de tipo letras y números, como la letra "A".

```
char unCaracter = 'A'; //Siempre comillas simples
```

|     | Numeros Enteros | Números reales con punto decimal | Valor único |
|-----|-----------------|----------------------------------|-------------|
| 8b  | byte            |                                  |             |
| 16b | short           |                                  | char ''     |
| 32b | Int             | float F                          |             |
| 64b | long L          | double                           |             |

- **Boolean:** datos de tipo lógico que sólo pueden ser verdadero o falso; por ejemplo, si es par o impar, positivo o negativo...

```
boolean par = true;  
boolean positivo = false;
```

**Referencia o complejos:** pueden estar formados por varias variables, otros datos complejos y/o funciones. Son objetos, y necesitan ser creados por el programador.

El más utilizado es el tipo de dato String, permite manejar textos y cadenas de texto. Ej. concatenar 2 tipos de datos "Hola" y "Fernando". El operador "+" permite concatenar cadenas y obtener el resultado esperado "Hola Fernando":

```
String str1= "Hola";  
String str2= "Fernando";  
String str3= str1+" "+str2; //str3 sería "Hola Fernando"
```

### e

#### Evento:

### f

### g

### h

## Java

### Herencia:

i

**Identificador:** nombres (únicos) que el programador da a los componentes que se están manejando en un programa, es decir, el nombre de las clases, métodos y variables.

#### Reglas:

- Nombres significativos.
- Se recomienda empezar por una letra, aunque después se pueden emplear números.
- No se permite el uso de caracteres especiales como: espacio en blanco, +, %, \*, etc.
- Comienzan con mayúsculas y cuando son nombres compuestos cada palabra comienza con mayúsculas. Ej. public class RecetaBizcocho.
- Java distingue entre mayúsculas y minúsculas. Ej. bizcocho es distinto que Bizcocho.
- Los nombres de constantes se escriben todo con mayúsculas y si el nombre es compuesto se usa el carácter de subrayado para separar las palabras. Ej. TIEMPO\_HORNEADO, pero para el nombre de la clase, métodos y atributos, no se usa el carácter de subrayado.
- No pueden coincidir con palabras reservadas/clave

j

**Java:** Lenguaje de programación orientado a objetos (POO). Los datos son representados por diferentes objetos que proporcionan una **biblioteca** estándar rica, testeada, fiable y reutilizable.

1. Código Java (.java)
2. Compilador Java JAVAC
3. Bytecode (.class)
4. Máquina virtual java JVM
5. Lenguaje máquina

Incluye características que protegen a los programadores contra errores no intencionados que pueden provocar problemas de integridad.

Se trata de un lenguaje **fuertemente tipado** y capaz de hacer validaciones en tiempo de compilación y ejecución.

Package **ejemplos;** → Paquete

//A continuación la clase → Comentario

class **Ejemplo1**{ → Clase

```
public static void main(String[]  
args) {  
    System.out.println("Hola Mundo");  
    System.out.println ("Esto es un  
ejemplo");  
}
```

→ Método  
main  
→ Bloque de  
sentencias

## Java

Proporciona una gestión de memoria automática. Cuando se crea un objeto, se asigna automáticamente espacio de memoria para ese objeto.

El recolector de basura Java realiza un seguimiento de cuáles son los objetos que la aplicación ya no necesita y recupera la memoria que ellos ocupan, evitando el problema de **fugas de memoria**.

Entorno multihilo, cada hilo (flujo de control del programa) representa un proceso individual dentro del programa, teniendo la capacidad de realizar múltiples tareas simultáneas.

Puede ejecutarse igualmente en cualquier otra plataforma o dispositivo. "write once, run anywhere".

**Java SE:** Standard Edition, Standalone, core, desktop, command line. JVM, JRE, JDK, SDK.

**Java EE:** Enterprise Edition, Service Side processing.

**API application programming interface.**

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/module-summary.html>

### Conceptos

|            | Qué es                   | Descripción                                                                                                 | Qué hace                                                                                       |
|------------|--------------------------|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>JVM</b> | Java Virtual Machine     | Máquina que ejecuta bytecode                                                                                | Compila un archivo .java, lo transforma en .class (bytecode)                                   |
| <b>JRE</b> | Java Runtime Environment | Entorno para ejecutar Java<br>JVM + Librerías estándar de Java (API)                                        | Lo que necesita un usuario final para ejecutar programas Java, pero no permite desarrollarlos. |
| <b>JDK</b> | Java Development Kit     | Kit para desarrollar en Java<br>JRE + herramientas para programar (compilador javac, javadoc, jar, jbd..)   | Lo que necesita un desarrollador para escribir, compilar y ejecutar aplicaciones Java.         |
| <b>SDK</b> | Software Development Kit | Término genérico que refiere a conjunto de herramientas para desarrollar software, en cualquier tecnología. | Ej. JDK es un SDK específico para Java.                                                        |
| <b>LTS</b> | Long-Term Support        | Versión con soporte extendido                                                                               | Actualizaciones de seguridad y correcciones de bugs por varios años (normalmente 8+ años).     |

**I**

**m**

**Método:** Comportamiento de una clase. Grupo de declaraciones o sentencias que realizan una tarea específica. Se puede llamar a un método tantas veces como lo necesitemos.

Debe contener el **tipo de retorno + nombre del método** (se recomienda que este siempre sea un verbo).

## Java

**Cabecera** de dicho método + **cuerpo** (grupo de sentencias) definido entre llaves "{}".

### Cabecera

- Modificadores (opcional).
- Tipo de retorno (void en caso de no retornar nada).
- Nombre: dar un nombre al método para poder ser invocado o usado cuando lo necesitemos; Dicho nombre se escribe en minúsculas, cuando tienen más de una palabra, la inicial de la segunda palabra se escribe en mayúscula.
- Lista de parámetros: Pasar información específica cuando sea necesario para la ejecución del método. Se separa por comas y se indica, por cada parámetro, el tipo y el nombre.

**Método main:** Método de entrada, el que da lugar al inicio del programa Java. Se llama siempre así y tiene que estar definido de la siguiente manera:

```
public static void main(String args[])
```

**Modificadores:** pueden ser adicional, precedidos de uno o varios.

**De acceso:** Especifica las restricciones de acceso o el alcance de una clase, método o variable.

- **public:** Cuando se declara un método como público, se está permitiendo que desde cualquier otra clase pueda referenciarse directamente.
- **private:** Si se declara un método como privado, no podrá ser invocado desde ninguna otra clase.
- **protected:** Los métodos así declarados son accesibles directamente desde la propia clase, de las subclases de esta (herencia) y las clases que se encuentran dentro del mismo paquete.

**De no acceso:** No cambian el alcance de la clase, método o variable, sino que les proporciona funciones adicionales

- **static:** Se usa para definir un método que está vinculado a una clase, en lugar de a su instancia, es decir, sirve para crear métodos que pertenecen a la clase y no a la instancia de la clase. ¿Esto qué implica?, que no es necesario crear una instancia u objeto de la clase para poder acceder o utilizar ese método, simplemente se pueden invocar por su nombre siempre que se use en la misma clase donde se defina o, si se usa desde otra clase, anteponiendo el nombre de la clase donde se ha creado: nombreClase.nombreMetodo.

## Java

- **final:** Un método declarado como final no puede ser sobrescrito por las subclases de la clase a la que pertenece (se explicará en el tema de herencia). Solo los métodos static pueden ser final.
- **abstract:** Define un método abstracto, es decir, es un método que NO tiene una implementación, sino que la implementación vendrá dada por las subclases de la clase a la que pertenece (se explicará en el tema de herencia). Es un método incompleto, no tiene cuerpo o implementación, por eso no tiene llaves "{}" y termina con punto y coma: `public abstract void saludar();`

## n

## o

**Operador:** Permiten realizar operaciones entre los tipos de datos básicos.

- **De asignación:** Asignar un valor a una variable, usa el signo igual (=).
- **Aritmético:** Realizar operaciones matemáticas. El operador suma (+) puede usarse también para cadenas de texto: `String saludo = a + b`, donde `a="Buenos "` y `b="días."`, ahora `saludo` es igual a `"Buenos días."`
- **Aritméticos combinados:** Combinar un operador aritmético con el operador de asignación (`+=`).
- **Unario:** Actúan sobre un único operando y sirven para incrementar y decrementar en uno el valor. (`++`, `--`) `Variable=++contador`
- **Relacional:** Realiza comparaciones entre tipos primitivos con resultado true o false. (`==`, `<`).
- **Lógicos:** Compara variables o expresiones y el resultado será true o false. (`||`, `&&`).
- **Lógico ternario:** Utiliza la lógica de la condicional, asigna un valor a una variable dependiendo del resultado de una comparación. En función del resultado de la expresión lógica, devolverá un valor u otro. `(?) (condición)?valor1:valor2;`

## p

**Palabras clave/reservadas:** definidas de manera exclusiva por el lenguaje de programación para un propósito específico.

Existen palabras reservadas para:

### Datos primitivos

| Palabra clave  | Descripción                            | Rango                      |
|----------------|----------------------------------------|----------------------------|
| <b>boolean</b> | true/false.                            |                            |
| <b>byte</b>    | número <b>entero</b> de 8 bits.        | -128 a 127 entero pequeño  |
| <b>char</b>    | dato <b>valor unico</b> de de 16 bits. | Entre comillas simples 'a' |



|               |                                              |                                                                           |
|---------------|----------------------------------------------|---------------------------------------------------------------------------|
| <b>float</b>  | número <b>real</b> en coma flotante 32 bits. | $\pm 3.4e38$ (menor precisión que double) Termina en <b>f</b> (ej. 3.14f) |
| <b>double</b> | número <b>real</b> en coma flotante 64 bits. | $\pm 1.7e308$                                                             |
| <b>short</b>  | número <b>entero</b> 16 bits.                | -32,768 a 32,767 entero mediano                                           |
| <b>int</b>    | número <b>entero</b> 32 bits.                | $\pm 2$ mil millones entero más común                                     |
| <b>long</b>   | número <b>entero</b> 64 bits.                | Agrega <b>L</b> al final (ej. 123L)                                       |

## Estructuras de control

|               | Palabra clave   | Descripción                                                                                                                     |
|---------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------|
| Condicionales | <b>if</b>       | instrucciones de control condicionales simples (if) o múltiples (else if).                                                      |
|               | <b>else</b>     | instrucción de control condicional doble que indica qué hacer en caso de que no se cumpla la condición del (if) o el (else if). |
|               | <b>switch</b>   | instrucción de control condicional múltiple.                                                                                    |
|               | <b>case</b>     | indicar cada caso de una instrucción de control (switch).                                                                       |
|               | <b>default</b>  | indica el caso por defecto de una instrucción de control (switch).                                                              |
| Bucles        | <b>for</b>      | escribir bucles que se repiten un número determinado de veces.                                                                  |
|               | <b>while</b>    | escribir bucles que se repiten mientras una condición sea cierta. La condición se valida al principio.                          |
|               | <b>do</b>       | escribir bucles (do while) que se repiten mientras una condición sea cierta. La condición se valida al final.                   |
| Salto/retorno | <b>break</b>    | interrumpe la ejecución de un bucle o de una instrucción.                                                                       |
|               | <b>continue</b> | interrumpe la ejecución de un bucle, pero permite seguir realizando interacciones.                                              |
|               | <b>return</b>   | interrumpe la ejecución del método y puede devolver un valor de retorno.                                                        |
| Excepciones   | <b>try</b>      | indica un bloque de código donde se quieren atrapar excepciones.                                                                |
|               | <b>catch</b>    | cláusula de un bloque (try) donde se especifica una excepción.                                                                  |
|               | <b>finally</b>  | permite especificar un bloque de código que siempre se ejecutará, se produzca o no una excepción.                               |
|               | <b>throw</b>    | permite lanzar una excepción.                                                                                                   |
|               | <b>throws</b>   | indica las excepciones que un método puede lanzar.                                                                              |

## Modificadores

| Palabra clave | Descripción                                                                                                                       |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>static</b> | indica que un elemento es único en una clase y que se puede acceder al él sin tener que crear una instancia u objeto de la clase. |

|                     |                                                                                                                              |
|---------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>public</b>       | indica que un elemento es accesible desde cualquier clase.                                                                   |
| <b>private</b>      | indica que un elemento es accesible únicamente desde la clase donde se ha definido.                                          |
| <b>abstract</b>     | definir clases y métodos abstractos.                                                                                         |
| <b>final</b>        | indica que una variable no se puede modificar, un método no se puede redefinir o una clase no se puede heredar.              |
| <b>protected</b>    | indica que un elemento es accesible desde la clase donde se ha definido, subclases de ella y otras clases del mismo paquete. |
| <b>transient</b>    | especificar que un atributo no sea persistente.                                                                              |
| <b>volatile</b>     | indica que el valor de un atributo que está siendo utilizado por varios hilos esté sincronizado.                             |
| <b>native</b>       | indica que un método está en un lenguaje de programación dependiente de la plataforma.                                       |
| <b>synchronized</b> | indica que un método o bloque de código es atómico.                                                                          |

## Estructura de datos

| Palabra clave     | Descripción                                                                              |
|-------------------|------------------------------------------------------------------------------------------|
| <b>class</b>      | definir una clase.                                                                       |
| <b>import</b>     | importa una clase que se encuentra en otro paquete o en la <a href="#">API de Java</a> . |
| <b>extends</b>    | permite indicar la clase padre de una clase.                                             |
| <b>interface</b>  | declarar una interfaz.                                                                   |
| <b>package</b>    | permite agrupar un conjunto de clases e interfaces.                                      |
| <b>implements</b> | definir la/s interfaces de una clase.                                                    |

## Otros

| Palabra clave     | Descripción                                                                          |
|-------------------|--------------------------------------------------------------------------------------|
| <b>super</b>      | permite invocar a un método o constructor de la superclase.                          |
| <b>const</b>      | usaba para la definición de constantes, pero ya no se utiliza, ahora se usa (final). |
| <b>this</b>       | referenciar al objeto e invocar al constructor.                                      |
| <b>new</b>        | crear un objeto nuevo de una clase.                                                  |
| <b>assert</b>     | afirmar que una condición es cierta.                                                 |
| <b>override</b>   | sobrescribir un método.                                                              |
| <b>instanceof</b> | permite saber si un objeto es una instancia de una clase concreta.                   |
| <b>void</b>       | dato vacío (sin valor).                                                              |
| <b>enum</b>       | definir tipos de datos enumerados.                                                   |
| <b>goto</b>       | se usaba para moverse de un punto a otro del código, pero ya no se utiliza.          |

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <b>strictfp</b> | indica que se tienen que utilizar cálculos en coma flotante estricto. |
|-----------------|-----------------------------------------------------------------------|

**Paquete:** forma de organizar las clases (indica la carpeta o directorio dónde está guardada la clase). Es la primera línea de código que debe estar en una clase Java.

Todas las clases que guardan cierta utilidad semejante se agrupan en un paquete. En una clase solo se puede indicar un paquete (cuando no se le da un nombre específico, el compilador crea uno por defecto llamado default package).

**Poliformismo:** concepto fundamental en la programación orientada a objetos, permite que objetos de diferentes clases sean tratados como objetos de una clase común.

### Precedencia de operadores

1. Paréntesis ( ) [ ] .
2. Operadores unarios: ++, --, !, (cast) new
3. Multiplicación \*, división /, módulo %
4. Suma + y resta -
5. Operadores relacionales: <, >, <=, >=
6. Igualdad: ==, !=
7. Operadores lógicos: && (AND), || (OR)
8. Operadores ternarios ?:
9. Asignación: =, +=, -=

**q**

**r**

**s**

**Sentencias:** unidad mínima de ejecución de un programa. Cada sentencia finaliza con punto y coma (;).

### Condicionales:

#### IF

Ejecutarán las instrucciones en función de la condición definida.

Condicional **if** (si...), y seguido indicamos la condición para que se ejecute la sentencia que va a continuación. Opcionalmente se puede añadir el bloque **else** (sentencias que sólo se realizan cuando no se cumple la condición if).

#### SWITCH..CASE

Instrucción de **múltiples vías** que permite ejecutar diferentes partes del código en función del valor de una expresión.

### Versión clásica:

El valor de la **expresión** debe ser de tipo `char`, `byte`, `short`, `int` o `String` (desde Java 7).

Cada **case** debe tener un valor único.

Después de cada caso se usa la palabra **break** para evitar que la ejecución "caiga" en el siguiente caso (*fall-through*).

Opcionalmente, se puede añadir el bloque **default** (solo se ejecuta si no se cumple ninguno de los casos anteriores).

```
switch (expresion) {
    case valor1:
        // Código a ejecutar
        break;
    case valor2:
        // Código a ejecutar
        break;
    default:
        // Código si no coincide ningún caso
}
```

### Versión flecha (Switch Expressions – Java 14+):

Puede **devolver un valor** directamente (se usa como expresión, no solo como bloque de código).

Usa la sintaxis `case valor ->` en vez de `case valor:`.

**No necesita break:** cada flecha es independiente.

**No hay fall-through:** nunca se pasa al siguiente caso por accidente.

Puede usarse para asignar el resultado a una variable o pasarlo directamente a un método como `System.out.println()`.

Requiere cubrir todos los casos posibles; si no, hay que poner **default**.

```
System.out.println(
    switch (numero) {
        case 1 -> "Lunes";
        case 2 -> "Martes";
        case 3 -> "Miércoles";
        case 4 -> "Jueves";
        case 5 -> "Viernes";
        case 6 -> "Sábado";
        case 7 -> "Domingo";
        default -> "Día inválido";
    })
```

## Java

```
    }  
    );
```

### Repetitivas o de bucle:

#### FOR

Se ejecutarán las sentencias repetidamente un número de veces, basándose en una condición. La condición se evaluará antes de cada iteración.

#### WHILE

Se ejecutarán y repetirán las sentencias mientras la condición sea verdadera.

Dentro del bucle, se han de actualizar los valores que se usan para la condición, si no se daría el caso de un bucle infinito. Si el resultado es false las instrucciones no se ejecutan y el programa seguirá ejecutándose por la siguiente instrucción a continuación del while.

#### DO WHILE

Igual que (WHILE), la diferencia es que primero ejecuta la condición y luego se evalúan las sentencias.

**Anidadas:** Incluyen instrucciones **if**, **for**, **while** o **do-while** unas dentro de otras, teniendo en cuenta que cada estructura que se anide debe estar cerrada dentro de la otra.

**System:** clase de la biblioteca estándar de Java (`java.lang.System`) provee métodos y campos para interactuar con el sistema donde corre el programa.

**System.in (input stream)**→ Flujo de entrada estándar (teclado). Normalmente se usa con un `Scanner` para leer datos

```
Scanner sc = new Scanner(System.in);  
int numero = sc.nextInt();
```

**System.out (output stream)**→ Flujo de salida estándar (consola). Muestra datos en pantalla.

**print()** → Imprime **sin** salto de línea.

**println()** → Imprime **con** salto de línea.

**printf()** → Imprime con formato (parecido a C).

t

u

## Java

### V

**Variable:** Objeto cuyo contenido puede variar durante el proceso de ejecución del algoritmo. Ej. datos para llevar a cabo una suma.

Declarar es identificar con un tipo y nombre a la variable. Es necesario declararlas antes de usarlas.

El tipo indica qué tipo de valores puede guardar.

El nombre permite acceder a su contenido y es necesario escribirlo en minúsculas.

```
<tipo dato> <nombre variable>;
```

Ej. Edad es un dato que puede variar durante el programa:

```
int edad;
```

Inicialización o asignación: se le da un valor por primera vez, antes de ser utilizada. A nivel de sintaxis se realiza usando el operador de asignación "=". Si no se inicia previamente, Java lo hace automáticamente.

Ej. valores numéricos Java les da un valor inicial de cero.

```
int edad = 0;
```

Asignar: previamente inicializada se le asigna un nuevo valor durante la ejecución. Dependiendo del tipo de variables se asignan de una manera u otra.

| Tipo variable | Descripción                            | Ejemplo                                 |
|---------------|----------------------------------------|-----------------------------------------|
| Numéricos     | se asignan directamente a la variable. | <code>int edad = 20;</code>             |
| Caracteres    | Usar la comilla simple. ' '            | <code>char genero = 'M';</code>         |
| Cadenas       | Usar comillas dobles. " "              | <code>String nombre = "Rodrigo";</code> |
| Booleanos     | Usar verdadero o falso (true o false). | <code>boolean casado = true;</code>     |

Ámbito: Define su alcance de uso, es decir en qué secciones de código está disponible. Fuera de este ámbito, una variable no existe.

|                    |                                                                                                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Local              | sólo están visibles dentro del bloque de código donde se han declarado (dentro de un método, un bucle, un condicional...), cuando éste finaliza, deja de existir. |
| Instancia o global | pertenecen a cada instancia concreta de la clase donde han sido declaradas y sólo pueden ser accesibles desde la propia instancia a la que pertenecen.            |

|                  |                                                                                                                                 |
|------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Clase o estática | pertenece a la clase, no a la instancia, y se puede acceder a ella desde cualquier parte de la clase donde han sido declaradas. |
|------------------|---------------------------------------------------------------------------------------------------------------------------------|